Secure Tunnel for UAV Telemetry with Post-Quantum Cryptography: Validated Protocol and Implementation Status

Anonymous Submission

***Abstract*—This manuscript presents an implementation-derived post-quantum secure tunnel for UAV telemetry and documents the currently validated system state. The protocol behavior is formalized in secure_tunnel_protocol_spec.md and cross-checked against two completed artifacts: the protocol conformance validation report and the NIST/Open Quantum Safe (OQS) cryptographic validation report. Validated findings cover the TCP handshake state machine, AEAD packet framing with authenticated headers, deterministic epoch/sequence nonce construction, commit-after-auth replay defense, and staged rekey state transitions. The manuscript reflects validated architecture and integration status only; performance and scheduler-effectiveness sections remain placeholders until benchmark campaigns are finalized.**

I. System Architecture

The architecture separates control-plane negotiation, encrypted data-plane forwarding, and telemetry/measurement collection. A TCP control channel performs authenticated handshake and rekey coordination, while the UDP data plane transports AEAD-protected MAVLink payloads.

The control plane combines in-band encrypted control messages with a legacy TCP JSON bridge for external orchestration. Rekey progression follows explicit prepare/commit/activate transitions managed by policy state. Cryptographic agility is implemented through a modular suite registry where canonical suite identity is key-handshake scoped (KEM+signature), and AEAD is selected as a separate runtime policy dimension. This enables scheduler-controlled algorithm agility with NIST-level filtering via suite metadata.

II. Secure Tunnel Protocol

The protocol description in this manuscript is implementation-derived and aligned with secure_tunnel_protocol_spec.md.

A. Handshake State Machine

A TCP-based handshake initializes each secure session. The server emits a signed ServerHello containing version, KEM/SIG identifiers, session identifier, challenge, KEM public key, and signature. The client verifies wire structure, transcript signature, and expected suite identity (downgrade rejection), then performs KEM encapsulation and returns ciphertext plus PSK-bound authentication and client key-confirmation material. The server verifies PSK authentication and client confirmation, performs KEM decapsulation, derives directional transport keys via HKDF-SHA256, and returns server key confirmation before data-plane activation.

B. AEAD Packet Format

The UDP wire format is header ‖ ciphertext_plus_tag, where the header is bound as AEAD associated data (AAD). Header fields encode protocol version, KEM/SIG identifier bytes, session identifier, sequence number, and epoch. The implementation does not transmit IV bytes on the wire; nonce material is reconstructed deterministically at receiver side.

C. Nonce Construction and Replay Protection

Nonce construction uses deterministic epoch/sequence encoding: one epoch byte concatenated with an 11-byte big-endian sequence basis, then adapted to cipher nonce length. Replay defense uses a bounded sliding-window bitmap with a pre-auth admissibility check and post-auth commit step. Window advancement occurs only after successful AEAD authentication, preventing unauthenticated replay-window poisoning.

D. Rekey Lifecycle

Rekeying follows a staged lifecycle (prepare, commit, activate) coordinated by control-plane state transitions. New sender/receiver contexts are staged first, then atomically activated with epoch progression after readiness conditions are met. Sequence-threshold safeguards and epoch wrap guards enforce key/nonce lifecycle boundaries; soft transition handling supports bounded old-context fallback during activation.

III. Cryptographic Foundations

The validated cryptographic baseline maps to current NIST PQC standards:

- FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM).
- FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA).
- FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA).

In the implemented handshake, KEM primitives establish the shared secret and signature primitives authenticate transcript material. Session key derivation uses HKDF-SHA256 with PSK/session/challenge/suite context and role-aware key binding. Runtime integrations use OQS KeyEncapsulation and Signature APIs for keypair generation, encapsulation/decapsulation, and sign/verify flows. The suite registry supports FIPS 203/204 runtime-default combinations (ML-KEM/ML-DSA) and includes FIPS 205 SLH-DSA variants

in the broader validated registry model; claims are restricted to behavior evidenced by the protocol conformance and NIST/OQS cryptographic validation artifacts.

IV. Implementation Details

Cryptographic operations are implemented through OQS Python bindings in the handshake runtime, including compatibility import paths for KeyEncapsulation/Signature objects and explicit failure paths when OQS support is unavailable. Data-plane protection is implemented with AEAD wrappers over a fixed authenticated header, deterministic nonce reconstruction, and replay-window commit-after-auth behavior. Control/runtime integration includes staged rekey context installation, activation-time context swap, and bounded previous-context fallback during soft transitions. Suite composition remains modular for scheduler-controlled agility, and no unvalidated security or performance claims are introduced.

V. Experimental Methodology

The benchmarking infrastructure is implemented and validated for readiness. Instrumentation pathways support latency, throughput, rekey/handshake timing, and power-oriented collection, while telemetry collectors cover power sensing, CPU/system context, and link/network statistics. Current methodology claims are limited to infrastructure availability and observability scope; benchmark execution is still in progress, and finalized quantitative outcomes are intentionally deferred.

VI. Results

Placeholder: Results are intentionally deferred until benchmark experiments complete.

VII. Performance Evaluation

Placeholder: Performance evaluation is intentionally deferred until benchmark experiments complete.

VIII. Scheduler Effectiveness

Placeholder: Scheduler effectiveness analysis is intentionally deferred until benchmark experiments complete.

IX. Conclusion

The protocol and implementation state presented here reflect validated architecture and cryptographic integration status, not final performance outcomes. Subsequent revisions will populate quantitative sections after completion of benchmark execution and analysis.